



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Pflichtenheft

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 30. Mai 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Lucas Neidahl	fertig
1.1	Lucas Neidahl	fertig
1.2	Lucas Neidahl	fertig
2	Lucas Neidahl	fertig
3	Luan Hyseni	fertig
3.1	Luan Hyseni	fertig
3.2	Luan Hyseni	fertig
4	Luan Hyseni	fertig
4.1	Luan Hyseni	fertig
4.2	Luan Hyseni	fertig
4.3	Luan Hyseni	fertig
5	Mohamed Sabri Jlizi	Fertig
5.1	Mohamed Sabri Jlizi	Fertig
5.2	Mohamed Sabri Jlizi	Fertig
5.3	Mohamed Sabri Jlizi	Fertig
5.4	Mohamed Sabri Jlizi	Fertig
5.5	Mohamed Sabri Jlizi	Fertig
6	Mohammad Khatib	fertig
6.1	Mohammad Khatib	fertig
6.2	Mohammad Khatib	fertig
6.3	Mohammad Khatib	fertig
6.4	Mohammad Khatib	fertig
6.5	Mohammad Khatib	fertig
7	Marvin Voigt	Fertig
8	Marvin Voigt	Fertig
9	Mohammed Alhalabi	Fertig

Inhaltsverzeichnis

1 Zielbestimmung	6
1.1 Motivation	6
1.2 Kriterien	7
1.2.1 Musskriterien	7
1.2.2 Sollkriterien	8
1.2.3 Kannkriterien	8
1.2.4 Abgrenzungskriterien	9
2 Produkteinsatz	10
2.1 Anwendungsbereiche	10
2.2 Zielgruppen	12
2.3 Betriebsbedingungen	12
3 Produktübersicht	14
4 Produktfunktionen	17
5 Produktdaten	22
5.1 Show-Konfigurationsdaten ⟨D10⟩	22
5.2 Drohnen-Daten ⟨D20⟩	22
5.3 Flugbahn-Daten ⟨D30⟩	23
5.4 Formation-Daten ⟨D40⟩	23
5.5 Simulationsdaten ⟨D50⟩	24
6 Nichtfunktionale Anforderungen	25
6.1 Funktionalität	25
6.1.1 Angemessenheit	25
6.1.2 Richtigkeit	25
6.1.3 Ineroperabilität	25
6.1.4 Ordnungsmäßigkeit	26
6.2 Sicherheit	26
6.2.1 Zuverlässigkeit	26
6.2.2 Reife	26
6.2.3 Fehlertoleranz	26

6.2.4	Wiederherstellbarkeit	27
6.3	Benutzbarkeit	27
6.3.1	Verständlichkeit	27
6.3.2	Erlernbarkeit	27
6.3.3	Bedienbarkeit	27
6.3.4	Zeitverhalten	28
6.3.5	Verbrauchsverhalten	28
6.4	Änderbarkeit	28
6.4.1	Analysierbarkeit	28
6.4.2	Modifizierbarkeit	28
6.4.3	Stabilität	29
6.4.4	Prüfbarkeit	29
6.4.5	Anpassbarkeit	29
6.4.6	Installierbarkeit	29
6.4.7	Konformität	29
6.4.8	Austauschbarkeit	30
6.5	Qualitätsanforderungen	30
7	Benutzeroberfläche/Schnittstellen	32
8	Technische Produktumgebung	35
8.1	Software	35
8.2	Hardware	35
8.3	Produktschnittstellen	36
9	Glossar	37

Abbildungsverzeichnis

3.1	Use-Case-Diagramm des SkyOrchestrator-Systems	14
3.2	Aktivitätsdiagramm – Konfiguration einer Show	15
7.1	Mockup der Benutzeroberfläche von SkyOrchestrator	34

1 Zielbestimmung

Ziel des Projekts SkyOrchestrator ist die Entwicklung einer Software zur Planung, Simulation und Visualisierung von Drohnenshows. Die Anwendung soll es Nutzern ermöglichen, Formationen und Bewegungsmuster für Drohnen zu gestalten und diese dann in dem Programm benutzerfreundlich zu visualisieren.

Das Projekt wird im Rahmen des Softwareentwicklungspraktikums (SEP) an der Technischen Universität Braunschweig erstellt. Wir verfolgen das Ziel, bis zum TDSE am 17. Juli 2025 einen funktionsfähigen Prototyp zu entwickeln, der als Plattform für zukünftige Erweiterungen dienen kann und diesen erfolgreich auf dem TDSE zu präsentieren.

Das Pflichtenheft dient als verbindliche Vertragsgrundlage für die Entwicklung von SkyOrchestrator. Es beschreibt wie und womit die Anforderungen technisch umgesetzt werden, definiert Anforderungen, Funktionen und Ziele des Projekts, um eine klare Orientierung für die Entwicklung zu schaffen, den Projektfortschritt zu steuern und es enthält detaillierte Systemanforderungen an unsere Software.

1.1 Motivation

Die Motivation für das Projekt ergibt sich aus der zunehmenden Suche nach Alternativen zu klassischen Feuerwerken. Traditionelles Feuerwerk hinterlässt viel Abfall, führt zu einer starken Lärmbelästigung gerade für Tiere und besitzt eine nicht unbeachtliche Verletzungsgefahr gerade für Kinder und Unerfahrene. Drohnenshows hingegen sind umweltfreundlicher, leise, emissionsfrei, sicherer und bieten trotzdem eine hohe kreative Flexibilität, durch das Steuern der einzelnen Drohnen. Allerdings sind sie bisher nur schwer zugänglich und müssen oft von externen Spezialisten bedient werden mit komplizierten Werkzeugen, deren Bedienung hohes technisches Fachwissen in Sachen wie Flugdynamik, Physik und Softwareentwicklung voraussetzt. Dies schafft eine hohe Einstiegsbarriere und somit ist es oft nicht praktikabel einsetzbar für Privatpersonen, kleinere Agenturen, Stadtverwaltungen oder Institutionen mit wenig Mitteln, die aber beispielsweise gerne Drohnenshows für Stadtfeste oder Events nutzen oder zumindest mal simulieren wollen. Genau hier setzt unser Projekt SkyOrchestrator an, denn durch eine intuitive Benutzeroberfläche, eine erweiterbare Architektur und individuelle visuelle Gestaltungsmöglichkeiten, welche dabei nicht nur die Bewegung, sondern auch die Steuerung der LED-Farben, Helligkeitsverläufe

und möglicher Effekte wie Blinken synchronisiert mit der Choreographie umfassen. Das alles soll sowohl für professionelle Eventplaner als auch kreative Laien es möglich machen solche kreativen Drohnenshows ohne Programmierkenntnisse zu bedienen, intuitiv zu simulieren und zu planen. Dabei soll die Software möglichst eine Echtzeit-Simulation zur Überprüfung der Drohnenshow bieten. Auch die Skalierbarkeit steht bei uns im Fokus, denn die Software soll nicht nur für kleine Shows einsetzbar sein, sondern auch größere Formationen mit vielen Drohnen ermöglichen.

1.2 Kriterien

In diesem Kapitel definieren wir die Anforderungen an die Software. Die Anforderungen werden in Musskriterien (unverzichtbar), Sollkriterien (erstrebenswerte Leistungen) und Kannkriterien (optionale Erweiterungen) gegliedert.

1.2.1 Musskriterien

Die Musskriterien stellen die unverzichtbaren Grundanforderungen dar, die für die Funktionsfähigkeit der Software zwingend erfüllt sein müssen. Sie definieren das Mindestmaß an Leistungsfähigkeit, Stabilität und Bedienbarkeit, das erforderlich ist, damit der SkyOrchestrator in realen Nutzungsszenarien einsetzbar ist.

⟨RM1⟩ Die Simulation erfolgt auf Basis der vom Nutzer eingegebenen Input Parametern, also die Anzahl und Formationsbedingungen der Drohnen muss bei der Simulierung beachtet werden

⟨RM2⟩ Visuelles Entwerfen von Formationen und Bewegungsabläufen über eine GUI muss durch den Nutzer möglich sein.

⟨RM3⟩ Es muss eine fest Timeline-Komponente zur Steuerung von Start, Ende, möglichen Formationswechseln, Lichteffekten und Landung vorhanden sein.

⟨RM4⟩ Die grundlegenden Planungs- und Simulationsfunktionen müssen so gestaltet sein, dass die Bedienung ohne Programmierkenntnisse möglich ist, also ohne einen Code-Befehl sollen Drohnen gespawnt, geflogen und grundlegende Eigenschaften der Drohnenshow verändert werden können.

⟨RM5⟩ Die Software muss eine ausführliche Fehlermeldung anzeigen, falls etwas schiefgeht (z.B. ungültige Eingaben oder Abstürze).

⟨RM6⟩ Die Anwendung muss auf vorgesehener Standardhardware (Windows 10 PC) lauffähig und bedienbar sein.

⟨RM7⟩ Nach dem Start muss die Anwendung im Hauptmenü landen und darf keine Fehlermeldungen erzeugen.

⟨RM8⟩ Die Speicherung und Wiederladen von Projekten in der Anwendung sollte möglich sein.

1.2.2 Sollkriterien

Die Sollkriterien betreffen Funktionen und Eigenschaften, die über die grundlegende Funktionsfähigkeit hinausgehen und die Benutzerfreundlichkeit sowie die professionelle Nutzbarkeit der Software erheblich verbessern.

⟨RS1⟩ Eine Bearbeitung und Synchronisation von LED-Farbeinstellungen und Helligkeitsverläufen sollte während der Simulation möglich sein.

⟨RS2⟩ Der Prototyp sollte in der Lage sein, die Simulation von Drohnenshows mit einer größeren Anzahl von Drohnen (z.B. 100 Drohnen) zu simulieren, um die Skalierbarkeit ansatzweise zu zeigen.

⟨RS3⟩ Die Farben und Lichteffekte sollten eingestellt werden können.

⟨RS4⟩ Die Simulation sollte möglichst flüssig und nahe an Echtzeit ablaufen in einer 3D-Umgebung.

⟨RS5⟩ Die GUI soll auf eine Standardbildschirmauflösung (z.B. 16:9) optimiert sein.

⟨RS6⟩ Die Software sollte konfigurierbare Optionen anbieten, z.B. Spracheinstellungen, Einheitenwahl oder Anzeigepräferenzen.

1.2.3 Kannkriterien

Die Kannkriterien umfassen optionale Erweiterungen, die nicht für den grundlegenden Betrieb notwendig sind, aber das Nutzererlebnis verbessern, die Flexibilität steigern oder neue Einsatzmöglichkeiten eröffnen können.

⟨RC1⟩ Import- und Exportfunktionen kann für echte reale Shows verfügbar sein .

⟨RC2⟩ Es könnte Kollisionserkennung der Drohnen mit Objekten und anderen Drohnen umgesetzt werden.

⟨RC3⟩ Einige vordefinierte Bewegungsmuster oder Formationen könnten als Vorlagen angeboten und erstellt werden.

⟨RC4⟩ Die Softwarearchitektur könnte modular und erweiterbar gestaltet sein, um zukünftige Funktionalitäten und Skalierbarkeit zu ermöglichen.

⟨RC5⟩ Es könnte eine Integration von Musik, um die Show mit Sounds zum synchronisieren der Show verfügbar sein.

⟨RC6⟩ Die Geschwindigkeit der Drohnen kann einstellbar sein

⟨RC7⟩ Eine Anzeige aller benutzten Drohnen während der Drohnenshow könnte im GUI angezeigt werden + ihren Status

1.2.4 Abgrenzungskriterien

Hiermit wird klar festgelegt, welche Funktionen, Rahmenbedingungen und Systemkomponenten bewusst nicht Teil des Projektumfangs sind. Es wird sich auf das fokussiert was in dem vorgegebenen Zeitrahmen mit unserem Vorwissen realistisch umsetzbar ist.

⟨RW1⟩ Die Software wird in diesem Projekt die Steuerung physischer Drohnen in der realen Welt nicht umsetzen. Es handelt sich um einen rein simulierten Prototypen.

⟨RW2⟩ Die Software wird keine komplexen realen Gegebenheiten wie Wind, Batteriestatus, GPS-Ungenauigkeit, Signalverlust, Luftfeuchtigkeit oder Temperatur bei der Simulation berücksichtigen.

⟨RW3⟩ Keine Definition unterschiedlicher Drohnentypen mit spezifischen Eigenschaften (Geschwindigkeit, Leuchtkraft, Größe usw).

⟨RW4⟩ Die Software wird keine automatische Optimierung der Drohnenbewegungen (z.B. kürzeste Flugwege) bieten.

⟨RW5⟩ Keine automatische Synchronisation mit Cloud-Speichern oder Online-Projektplattformen.

⟨RW6⟩ Es gibt keine physische Anleitung der Software

⟨RW7⟩ Es gibt keinen besonderen Administrationsmodus, welcher zusätzliche kritische Funktionen hat.

2 Produkteinsatz

Das folgende Kapitel definiert den geplanten Einsatzbereich von SkyOrchestrator. Es werden die verschiedenen Anwendungsgebiete beschrieben, für die das System entwickelt wird, sowie die spezifischen Zielgruppen identifiziert, die von der Software profitieren sollen. Darüber hinaus werden die technischen und organisatorischen Rahmenbedingungen dargestellt, unter denen SkyOrchestrator optimal funktioniert. Diese Informationen bilden die Grundlage für alle weiteren Entwicklungsentscheidungen und stellen sicher, dass das Endprodukt den Bedürfnissen der vorgesehenen Nutzer entspricht.

2.1 Anwendungsbereiche

Unsere Software SkyOrchestrator ist darauf ausgelegt, Drohnenshows benutzerfreundlich zu simulieren. Diese Drohnenshow-Simulationen können verschiedene Anwendungsbereiche haben, erstmal eignen sich Simulationen, um mehr über Drohnenshows und deren Choreographie zu lernen und für das Testen von Show-Ideen in einer virtuellen Umgebung. Schulen, Ausbildungsstätte und Hochschulen könnten unsere Software nutzen um ihren lernenden Drohnenchoreographie und Robotik praxisnah zu vermitteln. Man kann etwa aus der Simulation lernen und komplexe Flugrouten am PC zu planen, bevor sie in Realität umgesetzt werden. Die einfache Benutzeroberfläche erlaubt es den lernenden, Drohnenformationen ohne viel Vorwissen selbstständig zu entwerfen.

Unser Anliegen ist natürlich auch dass unsere Simulationssoftware mit der Zeit auch in der Realen Welt genutzt werden kann um echte Drohnenshows zu entwerfen nicht nur zu simulieren. Drohnenshows sollen als innovative und umweltschonende Alternative zum klassischem Feuerwerk dienen und Zuschauende begeistern. Das eignet sich besonders gut für Konzerte, Städtische Events und Festivals. Eventplaner können die Choreografien vorher mit unserer Software im 3D-Modell planen, simulieren und testen um somit Sicherheit und Funktionalität der Show zu gewährleisten. Dadurch werden solche Events sicherer und attraktiver gestaltet, solche Shows faszinieren schließlich alle Leute auch die sich vielleicht vor traditionellem Feuerwerk fürchten würden aufgrund von beispielsweise Lärmempfindlichkeit oder Psychischen Faktoren. Festivals könnten Drohnenschwärme auch für Hintergrund-Animationen einsetzen, mit unserer Software SkyOrchestrator kann man vorher am Bildschirm, als eine wichtige Planungsstufe, solche Formationen vorher testen und regelmäßig Drohnenshows als Publikumsmagnet kreieren.

Privatpersonen, kleine Vereine und Hobbyisten können auch unsere Software nutzen um selbst mehr über Drohenshows und choreographie zu lernen oder selbst Drohnenshows zu simulieren und mit wenigen Drohen kleine Shows zu entwerfen. Auch könnten Sie in der VR-Welt solche Drohnenshows erstellen, welche sich dann mit Hilfe von VR-Brillen in der virtuellen Welt bewundern lassen.

Nicht vorgesehen ist der Einsatz in militärischen Kontexten, in sicherheitskritischen Systemen oder zur präzisen Steuerung realer Drohnen im Echtzeitbetrieb, also für die präzise Flugphysik-Simulation, die das Neigen oder das Drehen der Drohnen miteinkalkuliert ist unser Projekt erstmal nicht geeignet. Ziel der Software ist eine visuell ansprechende und leicht verständliche Simulation, nicht die präzise Abbildung von Aerodynamik, Luftwiderstand oder physikalisch korrekter Flugbahnen. Auch ist unsere Software als PC-Anwendung gedacht nicht etwa als Mobile App oder als Website, da die oft stark begrenzte Rechenleistung nicht ausreicht, um Drohnenshows erfolgreich zu simulieren. Außerdem kann die Software zwar zur Planung genutzt werden, übernimmt aber keine rechtliche oder sicherheitstechnische Verantwortung für Flugzonen, Aufstiegsgenehmigungen oder Störquellenanalyse.

2.2 Zielgruppen

Primäre Zielgruppen:

- **Eventplaner und Veranstalter:** Personen, die für die Organisation von Events oder Festivals verantwortlich sind und nach innovativen Möglichkeiten suchen, ihre Veranstaltungen mit Drohnenshows zu bereichern.
- **Kreative und Künstler:** Designer, Choreographen oder Lichtkünstler, die visuelle Kunstwerke mit Drohnen gestalten möchten, ohne sich mit technischen Details auseinandersetzen zu wollen.
- **Bildungseinrichtungen:** Lehrer, Dozenten oder Kurs-Leiter, die die Software nutzen, um Schülern und Studenten die Grundlagen der Drohnenprogrammierung und -choreographie zu vermitteln.
- **Privat-Interessierte:** Technikbegeisterte oder Drohnen-Enthusiasten, die in ihrer Freizeit eigene Drohnenshows entwerfen und simulieren möchten, ohne technisches Vorwissen zu haben.
- **Kleine Agenturen:** Kleinere Event- oder Marketingagenturen, die Drohnenshows als Dienstleistung anbieten möchten, aber über das Budget oder die Möglichkeiten für Spezialisten und spezielles Werkzeug nicht verfügen.
- **Freizeitparks und Erlebniszentren:** Betreiber, die regelmäßig neue Showelemente konzipieren und diese vorab visuell testen und choreografieren wollen.
- **Studierende technischer Studiengänge:** Personen, die sich im Rahmen von Projekten, Seminaren oder Abschlussarbeiten mit Simulation und Visualisierung von Drohnenshows in der Theorie beschäftigen.

2.3 Betriebsbedingungen

Hier werden die unterschiedlichen Bedürfnisse und Anforderungen an das Produkt aufgelistet.

- mindestens Windows 10 (64 Bit) wird als Betriebssystem benötigt
- Ein leistungsfähiger Prozessor ist erforderlich (mindestens 4–8 Kerne, z.B. Intel i5/i7 oder vergleichbar)
- Arbeitsspeicher (RAM): Mindestens 16 GB RAM
- Mindestens 1 GB freier Festplattenspeicher für Installation und temporäre Projektdaten

- die Betriebszeit kann sehr Unterschiedlich sein und kann von wenigen Minuten für kurze Simulationen bis mehrere Stunden für komplexere Formationen bzw. langes anpassen einer Drohnenshow variieren.
- die Software wird von mindestens einer Person bedient, die mithilfe des Programms eine Simulation einer Drohnenshow erstellen kann. Für den realen Betrieb sind möglicherweise noch andere Mitarbeiter und Sicherheitsverantwortliche beziehungsweise Techniker erforderlich.
- Als Eingabegeräte werden Maus und Tastatur benötigt.

3 Produktübersicht

Das Diagramm zeigt die zentralen Interaktionsmöglichkeiten eines Nutzers mit dem System des SkyOrchestrator. Im Mittelpunkt stehen dabei drei wesentliche Ziele:

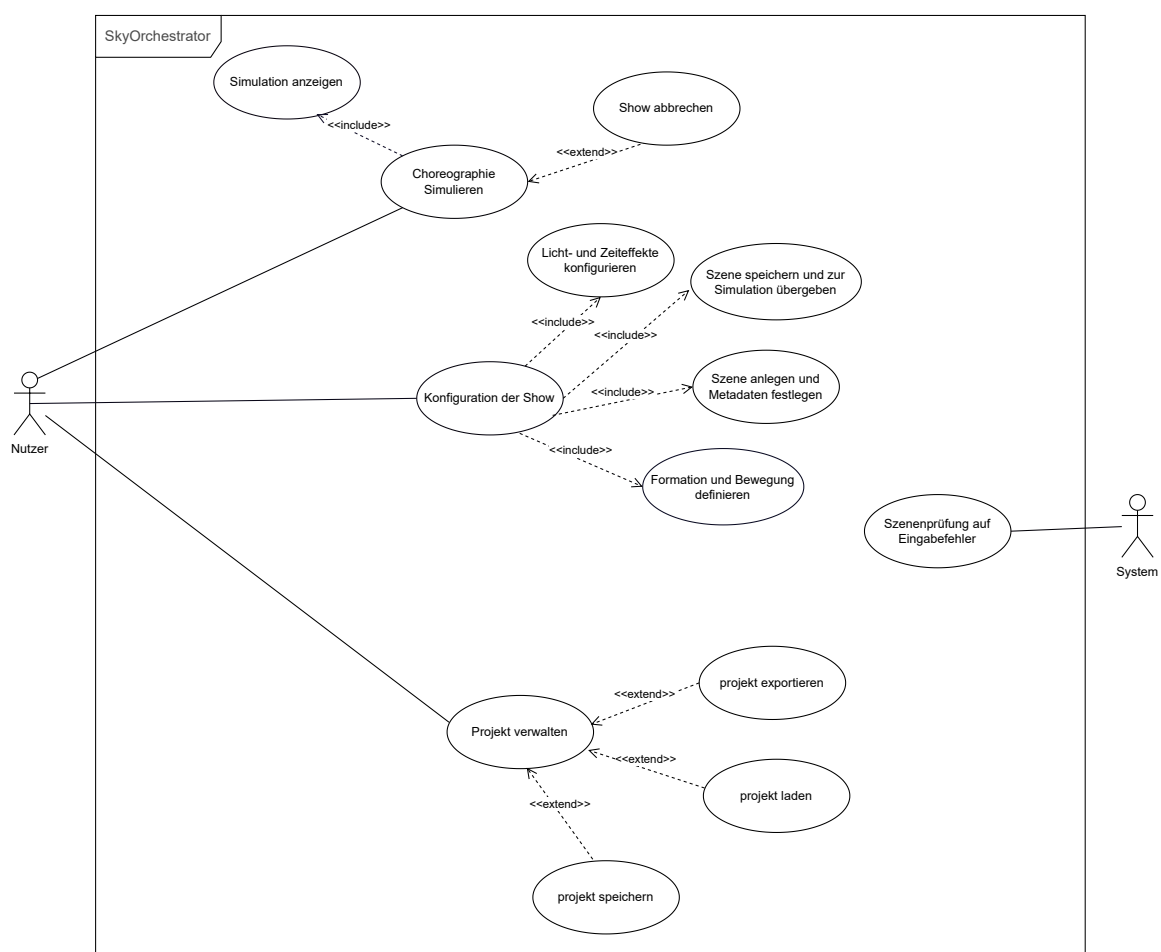


Abbildung 3.1: Use-Case-Diagramm des SkyOrchestrator-Systems

- **Konfiguration der Show:** Der Nutzer legt grundlegende Parameter der Drohnenshow fest, z.B. Anzahl der Drohnen, Startpositionen und zeitliche Rahmenbedingungen.
- **Choreographie simulieren:** Die zuvor konfigurierte Show kann als virtuelle Simulation abgespielt werden, um Bewegungsabläufe und Effekte zu überprüfen.

- **Projekt verwalten:** Der Nutzer hat die Möglichkeit, bestehende Projekte zu speichern, zu laden oder zu exportieren.
- **Szenenprüfung auf Eingabefehler:** Das System überprüft eine Szene auf fehlende Pflichtangaben oder ungültige Werte, bevor sie gespeichert oder simuliert wird.

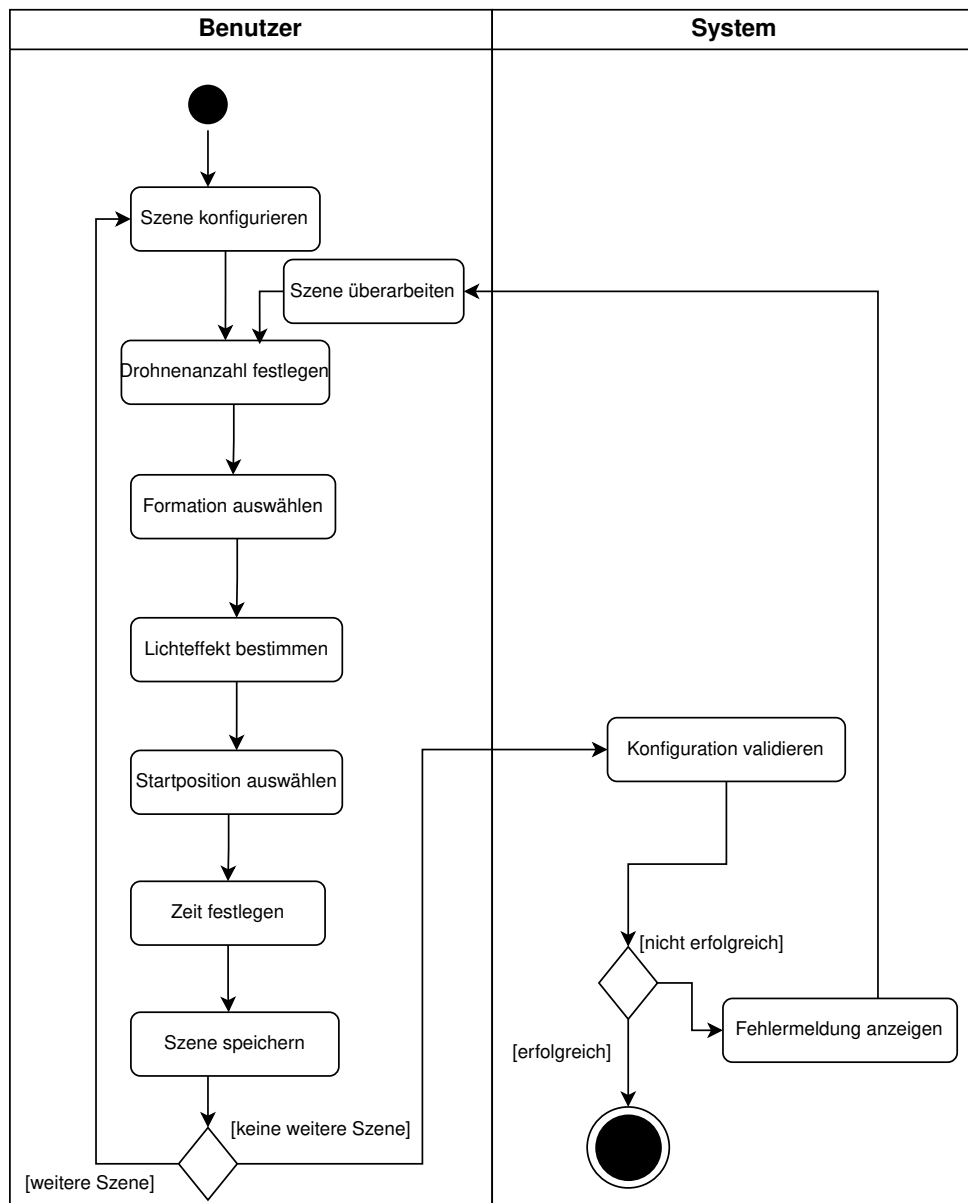


Abbildung 3.2: Aktivitätsdiagramm – Konfiguration einer Show

Das folgende Aktivitätsdiagramm beschreibt den typischen Ablauf der Konfiguration einer Drohnenshow. Der Konfigurationsprozess beginnt mit dem Anlegen einer ersten Szene. Danach legt der Benutzer die Anzahl der in dieser Szene eingesetzten Drohnen fest. Anschließend wird eine Formation gewählt, die die räumliche Anordnung der Drohnen beschreibt. Im weiteren Ver-

lauf werden Lichteffekte, Startposition und der zeitliche Ablauf in der Szene festgelegt. Ist die Konfiguration abgeschlossen, wird die Szene gespeichert. Nach dem Speichern entscheidet der Benutzer, ob eine weitere Szene zur Show hinzugefügt werden soll. Falls ja, wiederholt sich der Ablauf. Sobald keine weiteren Szenen mehr hinzugefügt werden, übergibt der Benutzer die vollständige Konfiguration an das System. Diese wird anschließend validiert. Ist die Validierung erfolgreich, so ist der Prozess abgeschlossen. Wenn die Validierung fehlschlägt, etwa weil ein erforderlicher Wert fehlt, muss die Szene erneut bearbeitet werden.

4 Produktfunktionen

Konfiguration der Show $\langle F10 \rangle$

Anwendungsfall: Der Benutzer plant die Inhalte einer Drohnenshow durch Erstellung, Bearbeitung und Verwaltung einzelner Szenen.

Anforderung: $\langle RM1 \rangle$, $\langle RM2 \rangle$, $\langle RM4 \rangle$, $\langle RM6 \rangle$, $\langle RM7 \rangle$

Ziel: Die Drohnenshow besteht aus konfigurierten Szenen mit definierter Bewegung, Beleuchtung und Zeitsteuerung, die zur Simulation übergeben werden können.

Vorbedingung: Die Anwendung ist gestartet und ein Projekt ist geöffnet oder wurde neu erstellt.

Nachbedingung Erfolg: Mindestens eine vollständige Szene wurde erfolgreich konfiguriert und gespeichert.

Nachbedingung Fehlschlag: Die Konfiguration bleibt unvollständig oder enthält Fehler, die vor der Simulation korrigiert werden müssen.

Akteure: Benutzer

Auslösendes Ereignis: Der Benutzer öffnet die Show-Konfiguration in der Benutzeroberfläche.

Beschreibung: Die Konfiguration erfolgt in mehreren logisch aufeinander folgenden Teilfunktionen:

- $\langle F11 \rangle$ Szene anlegen und Metadaten erfassen
- $\langle F12 \rangle$ Formation und Bewegung definieren
- $\langle F13 \rangle$ Licht- und Zeiteffekte konfigurieren
- $\langle F14 \rangle$ Szene speichern und zur Simulation übergeben

Szene anlegen und Metadaten erfassen $\langle F11 \rangle$

Anwendungsfall: Der Benutzer erstellt eine neue Szene und definiert grundlegende Metadaten wie Name, Dauer und Anzahl der Drohnen.

Anforderung: $\langle RM1 \rangle$, $\langle RM6 \rangle$

Ziel: Eine neue Szene mit korrekten Eingabedaten ist erstellt und zur weiteren Bearbeitung verfügbar.

Vorbedingung: Die Anwendung ist geöffnet, ein Projekt wurde gestartet oder geladen.

Nachbedingung Erfolg: Die Szene wird im Projekt gespeichert und ist bearbeitbar.

Nachbedingung Fehlschlag: Fehlerhafte oder unvollständige Angaben führen zu einer verständlichen Fehlermeldung.

Akteure: Benutzer

Auslösendes Ereignis: Der Benutzer klickt auf „Szene erstellen“ oder „neue Szene“.

Beschreibung: 1. Der Benutzer vergibt Name, Dauer und Beschreibung der Szene.

2. Die Anzahl der Drohnen wird festgelegt.

3. Das System erzeugt die Szene und speichert sie im Projekt.

Formation und Bewegung definieren $\langle F12 \rangle$

Anwendungsfall: Der Benutzer wählt eine Formation und legt Bewegungsmuster der Drohnen in der Szene fest.

Anforderung: $\langle RM1 \rangle$, $\langle RM2 \rangle$, $\langle RS2 \rangle$

Ziel: Die Formation und Bewegungsverläufe der Drohnen sind für die Szene definiert.

Vorbedingung: Eine Szene wurde erfolgreich erstellt.

Nachbedingung Erfolg: Die Bewegungen und Formationen sind gültig und gespeichert.

Nachbedingung Fehlschlag: Fehlerhafte Eingaben führen zu Rückmeldung an den Benutzer.

Akteure: Benutzer

Auslösendes Ereignis: Der Benutzer öffnet das Formations- oder Bewegungsmenü.

Beschreibung: 1. Der Benutzer wählt eine Formation oder erstellt eine neue.

2. Die Startpositionen und Flugpfade der Drohnen werden gesetzt.

3. Die Änderungen werden übernommen und geprüft.

Licht- und Zeiteffekte konfigurieren $\langle F13 \rangle$

Anwendungsfall: Der Benutzer legt Lichteffekte und zeitliche Steuerung fest.

Anforderung: $\langle RM4 \rangle$, $\langle RS1 \rangle$, $\langle RS4 \rangle$

Ziel: Die LED-Effekte und zeitlichen Abläufe der Szene sind vollständig definiert.

Vorbedingung: Die Formation und Drohnenanzahl sind bekannt.

Nachbedingung Erfolg: Alle Lichteinstellungen und Zeitpunkte sind gespeichert.

Nachbedingung Fehlschlag: Ungültige Effekte oder Zeitangaben führen zu einer Fehlermeldung.

Akteure: Benutzer

Auslösendes Ereignis: Der Benutzer öffnet das Licht- oder Zeitsteuerungsmenü.

Beschreibung: 1. Der Benutzer wählt Farben, Blinkverhalten und Übergänge.

2. Es erfolgt eine zeitliche Zuordnung über eine Zeitleiste.

3. Die Effekte und Zeitpunkte werden gespeichert und überprüft.

Szene speichern und zur Simulation übergeben $\langle F14 \rangle$

Anwendungsfall: Der Benutzer speichert die Szene und entscheidet über die Weiterbearbeitung oder Simulation.

Anforderung: $\langle RM6 \rangle$, $\langle RM7 \rangle$

Ziel: Die Szene ist gespeichert und bereit für Simulation oder Erweiterung.

Vorbedingung: Alle notwendigen Angaben zur Szene wurden vorgenommen.

Nachbedingung Erfolg: Die Szene ist gespeichert und wird ggf. an die Simulation übergeben.

Nachbedingung Fehlschlag: Die Szene enthält noch Eingabefehler oder ist unvollständig.

Akteure: Benutzer, System

Auslösendes Ereignis: Der Benutzer klickt auf „Szene speichern“ oder „Simulation starten“.

Beschreibung: 1. Das System prüft die Szene auf Vollständigkeit.

2. Bei Erfolg wird die Szene gespeichert.

3. Der Benutzer kann eine neue Szene starten oder die Simulation ($\langle F20 \rangle$) aufrufen.

Choreographie simulieren $\langle F20 \rangle$

Anwendungsfall: Eine zuvor konfigurierte Drohnenshow in der Simulation abspielen.

Anforderung: $\langle RM3 \rangle$, $\langle RM6 \rangle$, $\langle RM7 \rangle$, $\langle RS1 \rangle$

Ziel: Die geplante Choreographie wird in einer 3D-Umgebung möglichst flüssig und in Echtzeit simuliert.

Vorbedingung: Eine gültige Konfiguration mit mindestens einer gespeicherten Szene liegt vor.

Nachbedingung Erfolg: Die Simulation läuft fehlerfrei durch und stellt alle Bewegungen und Effekte korrekt dar.

Nachbedingung Fehlschlag: Die Simulation wird abgebrochen oder zeigt Fehler; es erfolgt eine Fehlermeldung.

Akteure: Benutzer, System

Auslösendes Ereignis: Der Benutzer startet die Simulation.

Beschreibung: 1. Der Benutzer wählt eine Konfiguration zur Simulation aus.

2. Das System berechnet Bewegungs- und Lichteffekte.

3. Die Choreographie wird visuell in Echtzeit wiedergegeben.
4. Bei Fehlern erfolgt eine Rückmeldung an den Benutzer.

Projekt verwalten $\langle F30 \rangle$

Anwendungsfall: Ein Projekt wird gespeichert, geladen oder exportiert.

Anforderung: $\langle RS3 \rangle$, $\langle RC1 \rangle$, $\langle RM7 \rangle$

Ziel: Der Benutzer kann eine bestehende Konfiguration speichern, wiederherstellen oder in ein Austauschformat exportieren.

Vorbedingung: Ein Projekt ist geöffnet oder wurde neu erstellt.

Nachbedingung Erfolg: Das Projekt wurde erfolgreich gespeichert, geladen oder exportiert.

Nachbedingung Fehlschlag: Der Vorgang schlägt fehl (z.B. durch ungültige Datei), es erfolgt eine entsprechende Fehlermeldung.

Akteure: Benutzer, System

Auslösendes Ereignis: Der Benutzer löst einen Speicher-, Lade- oder Exportvorgang aus.

Beschreibung: 1. Der Benutzer speichert die aktuelle Projektkonfiguration manuell oder automatisch.

2. Optional lädt der Benutzer ein zuvor gespeichertes Projekt.
3. Die Konfiguration kann als JSON oder Textdatei exportiert werden.
4. Bei Fehlern erfolgt eine Rückmeldung an den Benutzer.

Szenenprüfung auf Eingabefehler $\langle F40 \rangle$

Anwendungsfall: Der Benutzer überprüft eine Szene vor dem Speichern oder der Simulation. Das System erkennt fehlende Pflichtangaben oder ungültige Werte und zeigt sie gesammelt an.

Anforderung: $\langle RM6 \rangle$, $\langle RM7 \rangle$

Ziel: Fehlerhafte oder unvollständige Eingaben sollen dem Benutzer klar gemeldet werden, damit diese korrigiert werden können.

Vorbedingung: Mindestens eine Szene wurde erstellt.

Nachbedingung Erfolg: Alle Pflichtfelder sind korrekt ausgefüllt, die Szene ist speicherbar oder simulierbar.

Nachbedingung Fehlschlag: Das System zeigt eine Liste mit Fehlern an.

Akteure: Benutzer, System

Auslösendes Ereignis: Der Benutzer klickt auf „Szene prüfen“ oder versucht zu speichern/simulieren.

Beschreibung: 1. Das System prüft die Szene auf fehlende Pflichtangaben.

2. Ungültige Werte werden erkannt.
3. Eine strukturierte Fehlermeldungsliste wird angezeigt.
4. Der Benutzer kann die Fehler direkt korrigieren oder abbrechen.
5. Nach Korrektur kann die Szene gespeichert oder simuliert werden.

5 Produktdaten

Die langfristig zu speichernden Daten des SkyOrchestrator-Systems werden im Folgenden detailliert beschrieben. Diese Daten sind für die Planung, Simulation und Verwaltung von Drohnenshows essentiell.

5.1 Show-Konfigurationsdaten ⟨D10⟩

Daten zur Konfiguration einer Drohnenshow (max. 100 gespeicherte Shows):

- Show-ID (eindeutiger Identifikator)
- Show-Name (Benutzerdefinierter Name)
- Erstellungsdatum
- Letztes Änderungsdatum
- Autor/Ersteller
- Gesamtdauer der Show (in Sekunden)
- Anzahl der beteiligten Drohnen
- Show-Beschreibung (Freitext)
- Status (0 = Entwurf, 1 = fertig, 2 = archiviert)

5.2 Drohnen-Daten ⟨D20⟩

Konfiguration der einzelnen Drohnen (max. 100 Drohnen pro Show):

- Drohnen-ID (eindeutig innerhalb der Show)
- Startposition (X, Y, Z Koordinaten)
- LED-Konfiguration (Anzahl/Farbe der LEDs)
- Aktueller Status (0 = verfügbar, 1 = in Show, 2 = Wartung)

5.3 Flugbahn-Daten ⟨D30⟩

Daten der einzelnen Flugbahnen (max. 1.000 Wegpunkte pro Drohne):

- Drohnen-ID (Referenz)
- Wegpunkt-Nummer (Sequenz)
- Zielposition (X, Y, Z Koordinaten)
- Ankunftszeitpunkt (relativ zum Showstart)
- Bewegungsart (0 = linear, 1 = Kurve, 2 = Spirale etc.)
- LED-Farbe am wegpunkt
- LED-Helligkeit (0-100%)

5.4 Formation-Daten ⟨D40⟩

Definition von Drohnenformationen (max. 50 Formationen pro Show):

- Formation-ID
- Formation-Name
- Typ (0 = statisch, 1 = dynamisch)
- Liste der beteiligten Drohnen-IDs
- Relative Positionen der Drohnen zueinander
- Zeitpunkt der Formation (relativ zum Showstart)
- Dauer der Formation

5.5 Simulationsdaten 〈D50〉

Ergebnisse der Show-Simulation:

- Simulations-ID
- Referenzierte Show-ID
- Simulationsdatum
- Dauer der Simulation
- Durchschnittliche Framerate (FPS)
- Anzahl der erkannten Kollisionsrisiken
- Speicherplatz der aufgezeichneten Simulation

Diese Datenstrukturen ermöglichen die vollständige Beschreibung einer Drohnenshow von der Planung bis zur Simulation. Die Daten werden primär im JSON-Format gespeichert, um eine einfache Austauschbarkeit mit realen Drohnensystemen zu gewährleisten.

6 Nichtfunktionale Anforderungen

Dieses Kapitel definiert die Qualitätsanforderungen für das zu entwickelnde Produkt, einschließlich der jeweiligen Qualitätsstufen. Die wichtigsten Qualitätsmerkmale werden anschließend operationalisiert, also in konkrete Produktspezifikationen übersetzt, sofern sie nicht bereits als allgemeine Standards oder Normen vorliegen.

6.1 Funktionalität

Produktqualität	sehr gut	gut	normal	nicht relevant
Angemessenheit		x		
Richtigkeit	x			
Interoperabilität		x		
Ordnungsmäßigkeit	x			

6.1.1 Angemessenheit

Die Software erfüllt praxisnahe Anforderungen zur Planung von Drohnenshows. Sie bietet intuitive Benutzerführung und grundlegende Funktionen wie Flugplanung, Lichtsteuerung und Simulation.

6.1.2 Richtigkeit

Die Berechnungen der Flugpfade und zeitlichen Abläufe basieren auf physikalischen Simulationen mittels PyBullet. Dies gewährleistet eine realitätsnahe Darstellung der Bewegungen. Allerdings berücksichtigt die Simulation aktuell keine automatische Kollisionsvermeidung, sodass sie für die Planung realer Drohnenflüge nicht direkt geeignet ist.

6.1.3 Ineroperabilität

Durch den geplanten Export in XML und die Nutzung offener Formate ist eine gute Anbindung an reale Drohnensysteme sowie externe Tools gewährleistet.

6.1.4 Ordnungsmäßigkeit

Alle Projektvorgaben der Universität und der SEP-Richtlinien werden eingehalten. Die Dokumentation erfolgt vollständig und strukturiert, inklusive Testspezifikationen und Pflichtenheft.

6.2 Sicherheit

Produktqualität	sehr gut	gut	normal	nicht relevant
Zuverlässigkeit				x
Reife	x			
Fehlertoleranz			x	
Wiederherstellbarkeit	x			

6.2.1 Zuverlässigkeit

Diese Kategorie wurde als weniger relevant eingestuft, da das Projekt keine sicherheitskritische Software für den realen Flugbetrieb implementiert. Die Anwendung dient ausschließlich der Simulation und Planung und darf nicht für die Steuerung realer Drohnen verwendet werden. Dennoch ist auf Zuverlässigkeit im Sinne der Softwarestabilität und Lauffähigkeit zu achten.

6.2.2 Reife

Die sehr gute Bewertung ergibt sich daraus, dass das Projekt durchdacht geplant, kontinuierlich getestet und vollständig dokumentiert wird. Die Verwendung bewährter Technologien (z.B. Gitlab für Versionierung, PyBullet für die Simulation) trägt zur Stabilität und Robustheit bei.

6.2.3 Fehlertoleranz

Hier wurde normal bewertet, da grundlegende Fehlerbehandlungsmechanismen integriert sind (z.B. für Benutzereingaben oder Dateiformate), aber keine hochentwickelte Redundanz oder automatische Fehlerkompensation vorgesehen ist – was bei einer Simulationssoftware jedoch angemessen ist.

6.2.4 Wiederherstellbarkeit

Die Wiederherstellbarkeit wird als gut bewertet. Die Anwendung speichert Benutzereingaben regelmäßig automatisch in temporären Sicherungsdateien (Auto-Save), um Datenverlust bei Systemabstürzen oder unerwarteten Fehlern zu vermeiden. Zusätzlich wird beim manuellen Speichern eine Sicherungskopie der letzten Version angelegt. Sollte das Programm abstürzen, wird dem Nutzer beim nächsten Start die Wiederherstellung der letzten automatisch gespeicherten Projektversion angeboten.

6.3 Benutzbarkeit

Produktqualität	sehr gut	gut	normal	nicht relevant
Verständlichkeit		x		
Erlernbarkeit			x	
Bedienbarkeit		x		
Effizienz				
Zeitverhalten		x		
Verbrauchsverhalten		x		

6.3.1 Verständlichkeit

Die Benutzeroberfläche wird so gestaltet, dass die Funktionen und Abläufe nachvollziehbar sind, auch ohne technische Dokumentation. Dennoch besteht hier Spielraum für Verbesserung, z.B. durch kontextbezogene Hilfetexte oder Tooltips.

6.3.2 Erlernbarkeit

Da die Anwendung einige komplexe Funktionen wie Flugrouten-Definition im 3D-Raum oder Lichtanimationen enthält, wird eine gewisse Einarbeitungszeit erforderlich sein. Daher wurde dieser Aspekt als „normal“ bewertet.

6.3.3 Bedienbarkeit

Die GUI basiert auf einer klaren Struktur mit logischen Workflows, was eine intuitive Bedienung ermöglicht. Die Bewertung „gut“ spiegelt das Ziel wider, eine effektive und unkomplizierte Steuerung zu bieten.

6.3.4 Zeitverhalten

Die Simulationen sollen in möglichst flüssiger Echtzeit ablaufen, was durch geeignete Optimierungen (z.B. reduzierte Drohnenanzahl bei Vorschauen) unterstützt wird.

6.3.5 Verbrauchsverhalten

Da es sich primär um eine Desktop-Anwendung handelt, die auf modernen Rechnern ausgeführt wird, spielt der Ressourcenverbrauch nur eine untergeordnete Rolle.

6.4 Änderbarkeit

Produktqualität	sehr gut	gut	normal	nicht relevant
Analysierbarkeit		x		
Modifizierbarkeit		x		
Stabilität	x			
Prüfbarkeit		x		
Übertragbarkeit				
Anpassbarkeit				x
Installierbarkeit		x		
Konformität		x		
Austauschbarkeit			x	

6.4.1 Analysierbarkeit

Der Quellcode wird nach gängigen Konventionen strukturiert, mit ausführlichen Kommentaren und Docstrings versehen, sodass sich auch neue Entwickler:innen schnell einarbeiten und bestehende Strukturen verstehen können.

6.4.2 Modifizierbarkeit

Durch eine modulare Architektur und klar definierte Schnittstellen können einzelne Komponenten einfach angepasst oder erweitert werden, ohne tiefgreifende Änderungen am Gesamtsystem vornehmen zu müssen.

6.4.3 Stabilität

Die Software wird in mehreren Iterationen getestet, wobei besonderer Fokus auf das zuverlässige Verhalten bei Änderungen gelegt wird. Unit-Tests und kontinuierliches Bugfixing tragen zur Stabilität bei.

6.4.4 Prüfbarkeit

Im Projektverlauf ist eine Testphase vorgesehen, in der die Software systematisch überprüft wird. Ergänzend dazu werden Testspezifikationen und Testprotokolle erstellt, um die Nachvollziehbarkeit und Dokumentation der Testergebnisse sicherzustellen.

6.4.5 Anpassbarkeit

Die Software ist so konzipiert, dass sie auf verschiedenen Systemumgebungen flexibel einsetzbar ist. Primär wird sie unter Windows 10 und 11 getestet, ist aber durch die Nutzung plattformunabhängiger Technologien (z.B. Python, Tkinter) grundsätzlich auch auf gängigen Linux-Distributionen lauffähig. Die Konfiguration von Hardwareparametern (z.B. Auflösung, GPU-Leistung) erfolgt dynamisch, sodass eine Anpassung an leistungstärkere oder schwächere Systeme möglich ist. Einschränkungen bestehen lediglich bei sehr leistungsschwacher Hardware, hier kann es zu reduzierter Simulationsqualität kommen. Eine plattformübergreifende Erweiterung (z.B. MacOS) wäre perspektivisch mit vertretbarem Aufwand realisierbar.

6.4.6 Installierbarkeit

Die Anwendung wird so entwickelt, dass sie sich auf Zielsystemen mit Python-Umgebung problemlos installieren und starten lässt. Eine Anleitung unterstützt diesen Prozess.

6.4.7 Konformität

Die Software wird nach den Vorgaben der Universität und den Richtlinien des SEP entwickelt. Beim Programmieren halten wir uns an den PEP8-Standard für sauberen Python-Code. Für die Versionsverwaltung wird Git verwendet. Die verwendeten Datenformate wie JSON und XML sind allgemein verbreitet und sorgen dafür, dass die Software gut mit anderen Systemen zusammenarbeiten kann. Auch bei der Benutzeroberfläche achten wir auf gute Lesbarkeit und einfache Bedienung, obwohl keine speziellen Standards zur Barrierefreiheit umgesetzt werden.

6.4.8 Austauschbarkeit

Da das System eine sehr spezifische Anwendung erfüllt und nicht als austauschbares Modul innerhalb anderer Systeme gedacht ist, ist dieser Aspekt nicht relevant.

6.5 Qualitätsanforderungen

In diesem Abschnitt werden die wichtigsten Qualitätsmerkmale für SkyOrchestrator in konkrete Anforderungen übersetzt. Ziel ist es, messbare Kriterien festzulegen, mit denen sich Benutzerfreundlichkeit, Leistung und Zuverlässigkeit der Software bewerten lassen.

- $\langle Q10 \rangle$.
Learnability: Neue Nutzer:innen sollen die Grundfunktionen (Choreografie erstellen, Simulation starten) binnen 15.Minuten ohne Anleitung beherrschen.
- $\langle Q20 \rangle$.
Effizienz: Häufig genutzte Funktionen (z.B. Drohnen hinzufügen, Formation wechseln) müssen mit maximal zwei Klicks erreichbar sein.
- $\langle Q30 \rangle$.
Fehlertoleranz: Die UI unterstützt Eingabefehler durch automatische Validierung, Vorschläge und Korrekturhinweise.
- $\langle Q40 \rangle$.
Ressourcenverbrauch: Der Speicherverbrauch der Anwendung darf während einer Simulation mit 50 Drohnen 1 GB RAM nicht überschreiten.
- $\langle Q50 \rangle$.
Dokumentation: Kontextsensitive Hilfetexte und Tooltips sind für alle Hauptfunktionen verfügbar.
- $\langle Q60 \rangle$.
Reaktionszeit UI: Alle Reaktionen auf Benutzeraktionen außerhalb der Simulation (z.B. Szenen anlegen, Parameter ändern) dürfen 1 Sekunde nicht überschreiten.
- $\langle Q70 \rangle$.
Plattformkonformität: Die Software muss ohne Codeänderungen auf Python 3.11+ unter Windows 10/11 starten und lauffähig sein.

- $\langle Q80 \rangle$.

Fehlertoleranz: Ungültige Benutzereingaben (z.B. fehlerhafte Koordinaten) müssen durch Validierungsmechanismen erkannt werden, und eine verständliche Fehlermeldung innerhalb von 1 Sekunde angezeigt werden.

7 Benutzeroberfläche/Schnittstellen

Die Benutzeroberfläche der Software *SkyOrchestrator* ist zentraler Bestandteil der Interaktion zwischen Anwender und System. Sie ist modular aufgebaut und soll eine intuitive Konfiguration, Planung und Simulation komplexer Drohnenshows ermöglichen. Ziel ist es, auch technisch weniger versierten Nutzern die Gestaltung von Bewegungsprofilen, Lichtanimationen und Ablaufsteuerung mehrerer Drohnen zu ermöglichen.

Allgemeine Anforderungen an die Benutzeroberfläche $\langle UI10 \rangle$

Die Benutzeroberfläche der Software *SkyOrchestrator* ist auf eine einfache, intuitive Bedienung ausgelegt. Ziel ist es, dem Nutzer eine möglichst klare Steuerung der Show-Konfiguration, Zeitplanung und Simulation zu ermöglichen.

Die wichtigsten Designprinzipien sind:

- Klare Trennung von Steuer- und Anzeigeelementen
- Übersichtliche Gliederung nach Funktionsbereichen (Toolbar, Showanzeige, Zeitleiste, Drohnenübersicht)
- Intuitive Mausbedienung und direkte Manipulation (z.B. über Buttons und Auswahlfelder)
- Responsives Layout

Fensterlayout $\langle UI20 \rangle$

Die GUI gliedert sich in mehrere funktionale Bereiche siehe Abbildung 7.1

- **Toolbar (links):** Zugriff auf Startseite, Projekteinstellungen und Systemkonfiguration.
- **Simulationsfenster (zentral):** Hauptanzeige für die Show-Visualisierung, in der Formationen, Bewegungen und Lichteffekte in Echtzeit dargestellt werden.
- **Seitensteuerfeld (rechts):**
 - Anzeige und Steuerung des Timers (Start/Pause)
 - Timeline-basierte Effektauslösung (z.B. Lichtwechsel, Formationsänderung)
 - Auswahl von Drohnenanzahl, Formation, Lichtfarbe und Musik

- **Drohnenübersicht (unten):**

- Anzeige aller Drohnen-IDs mit Status und Koordinaten
- Direkte Auswahl einzelner Drohnen zur gezielten Bearbeitung
- Scrollfunktion bei hoher Drohnenanzahl

Dialogstruktur $\langle UI30 \rangle$

Die gesamte Steuerung erfolgt über Mausbedienung und optional über Tastaturkürzel. Interaktive Felder wie Dropdowns, Schieberegler und Zeitachsen ermöglichen schnelle Änderungen ohne komplexe Menüs. Die Timeline erlaubt die zeitgenaue Platzierung von Effekten und Formationswechseln, was besonders bei der Musik-Synchronisation relevant ist.

Rollen und Zugriffsrechte $\langle UI40 \rangle$

Aktuell ist die Software auf eine Benutzerrolle ausgelegt. Diese hat vollständigen Zugriff auf alle Funktionen der Anwendung. Zukünftig können weitere Rollen wie z.B. Techniker*innen mit eingeschränkten Rechten ergänzt werden.

Tabelle: Benutzerrollen im aktuellen System

Im aktuellen System ist eine primäre Benutzerrolle vorgesehen. Weitere Rollen können im Zuge einer späteren Version ergänzt werden.

Rolle	Rechte	Benutzeroberfläche
Show-Ersteller*in	$\langle F10 \rangle$, $\langle F20 \rangle$, $\langle F30 \rangle$	Vollzugriff auf alle GUI-Bereiche

Visualisierung (Mockup) $\langle UI50 \rangle$

Die nachfolgende Abbildung zeigt ein Mockup der Benutzeroberfläche. Es dient der Veranschaulichung der geplanten Anordnung der Steuerelemente.

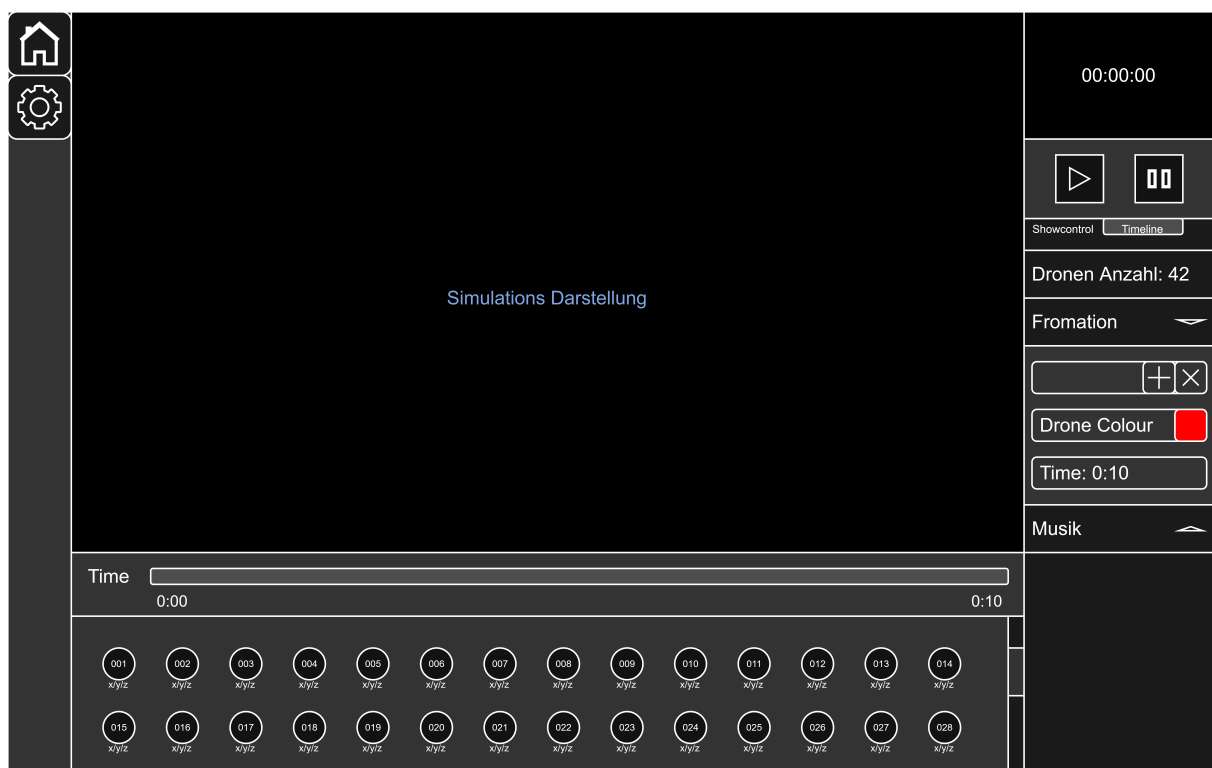


Abbildung 7.1: Mockup der Benutzeroberfläche von SkyOrchestrator

8 Technische Produktumgebung

In diesem Kapitel wird die technische Grundlage der Anwendung *SkyOrchestrator* beschrieben. Dazu gehört sowohl die eingesetzte Software, als auch die zugrunde liegende Hardware, auf der die Entwicklung und Tests durchgeführt werden. Zusätzlich werden die vorhandenen und geplanten Schnittstellen des Produkts erläutert.

8.1 Software

Für die Erstellung und Ausführung der Anwendung werden die nachfolgenden Softwaresysteme genutzt.

- **Client-Betriebssystem:** Windows 10 / Windows 11
- **Simulations-Engine:** PyBullet (wird in die Anwendung eingebettet)
- **Programmiersprache:** Python 3.13 (hauptsächlich), alternativ C oder C++
- **Grafische Benutzeroberfläche:** Tkinter (vollständig in deutscher Sprache)
- **Sonstige Anforderungen:** Eingesetzte Simulations-Tools und Benutzeroberflächen sollen, wenn möglich, in deutscher Sprache verfügbar sein.

Die Anwendung wird unter Windows 10/11 entwickelt und getestet. PyBullet wird als Simulationsbibliothek in die Anwendung integriert, um realistische physikalische Abläufe der Drohnenshow darzustellen. Die Benutzeroberfläche ist auf eine deutschsprachige Zielgruppe ausgelegt und wird entsprechend lokalisiert umgesetzt.

8.2 Hardware

Die Anwendung wird auf einem Referenzsystem entwickelt und getestet, das die folgenden Hardwareanforderungen erfüllt:

- **Prozessor:** Intel Core i7-7700K @ 4,20 GHz

- **Arbeitsspeicher:** 16 GB RAM
- **Grafikkarte:** NVIDIA GeForce RTX 2080
- **Betriebssystem:** Windows 10 Pro (64 Bit)

Diese Konfiguration gewährleistet eine zuverlässige Ausführung der physikalischen Simulation sowie eine flüssige grafische Darstellung der geplanten Drohnenshows. Für den produktiven Einsatz wird eine vergleichbare oder leistungsfähigere Hardware empfohlen.

Hinweis zur Skalierbarkeit und Mindestanforderungen: Das System kann auch auf weniger leistungsfähiger Hardware betrieben werden, allerdings mit reduzierter Simulationsgenauigkeit oder vereinfachter grafischer Darstellung. Die minimalen Systemanforderungen orientieren sich an den Anforderungen der verwendeten Simulationsbibliothek PyBullet:

- **Prozessor:** Quad-Core CPU ab 2,5 GHz
- **Arbeitsspeicher:** 8 GB RAM
- **Grafikkarte:**
 - NVIDIA GTX 680 oder
 - AMD Radeon HD 7970
- **Hinweis:** PyBullet unterstützt keine GPUs, die ausschließlich OpenCL verwenden.
- **Betriebssystem:** Windows 10 (64 Bit)

8.3 Produktschnittstellen

Da die Software *SkyOrchestrator* in der aktuellen Entwicklungsphase als eigenständige Anwendung konzipiert ist, werden derzeit keine direkten Schnittstellen zu externen Systemen oder Produkten benötigt. Geplant ist jedoch die Möglichkeit, die erstellten Drohnenshows in standardisierten Formaten wie JSON oder XML zu speichern und bei Bedarf wieder zu laden.

9 Glossar

PyBullet Physik-Engine zur Simulation physikalischer Abläufe in Echtzeit. Wird zur Darstellung der Drohnenshow verwendet.

JSON Textbasiertes Format zur strukturierten Speicherung von Show- und Konfigurationsdaten.

XML Textbasiertes Austauschformat zur Speicherung und zum Export von Shows.

Timeline Zeitstrahl-Komponente in der Benutzeroberfläche zur Steuerung der Show

Mockup Entwurfsgrafik einer Benutzeroberfläche zur Veranschaulichung von Design und Layout.

Simulation Die virtuelle Darstellung der geplanten Drohnenshow

Formation Räumliche Anordnung mehrerer Drohnen innerhalb einer Szene.

Wegpunkt Ein Zielpunkt, den eine Drohne im Rahmen ihrer Flugbahn ansteuert.

Szenen Einzelne Abschnitte innerhalb einer Drohnenshow mit spezifischen Konfigurationen für Drohnen, Licht und Timing.

Produktdaten Daten, die zur Planung, Ausführung und Simulation einer Drohnenshow notwendig sind.

SkyOrchestrator Name der entwickelten Simulationssoftware zur Planung und Darstellung von Drohnenshows.

SEP Softwareentwicklungspraktikum der TU Braunschweig, im Rahmen dessen das Projekt umgesetzt wird.

TDSE Technologietag der Studierendenprojekte – Präsentationstermin des SEP-Projekts.

GUI Grafische Benutzeroberfläche der Software zur Bedienung von Konfiguration, Simulation und Steuerung.

Toolbar Menübereich der GUI mit zentralen Steuerungselementen (z.B. Start, Speichern, Simulation starten).

Effekt Eine visuelle Aktion in der Simulation, z.B. Farbwechsel oder Blinken der Drohnen-LEDs.

Konfiguration Zusammenspiel aller Einstellungen

Use-Case-Diagramm Diagrammtyp der UML zur Beschreibung der Benutzerinteraktionen mit einem System. Es zeigt auf, welche Funktionen ein System aus Sicht eines externen Nutzers anbietet.

Aktivitätsdiagramm UML-Diagramm zur Darstellung des logischen Ablaufs von Aktivitäten und Entscheidungen innerhalb eines Prozesses oder Systems.

Kollisionserkennung Funktionalität zur automatischen Erkennung von möglichen Kollisionen zwischen Drohnen oder mit statischen Objekten innerhalb der Simulation.

GitLab Webbasierte Plattform zur Versionsverwaltung von Quellcode mit Funktionen für CI/CD, Issue Tracking und Teamarbeit.